KEMENTERIAN PENDIDIKAN DAN KEBUDAYAAN
UNIVERSITAS TADULAKO
FAKULTAS TEKNIK — JURUSAN TEKNOLOGI INFORMASI
PROGRAM STUDI S1 TEKNIK INFORMATIKA
Kampus Bumi Tadulako Tondo, Jl. Soekarno Hatta Km. 9, Palu — Sulawesi Tengah
Telp. (0451) 422611 – 422355 | Fax. (0451) 422844 | Email: untad@untad.ac.id

MODUL PRAKTIKUM — MATA KULIAH REKAYASA API (F09252013)

Mata Kuliah	Rekayasa API	**Kode Mata Kuliah**	F09252013
Program Studi	S1 Teknik Informatika	**Semester**	3 (Tiga)
Beban Studi	4 SKS	**Pertemuan Ke-**	-
Judul Praktikum	**Praktikum 3 — Membangun RESTful API from Scratch dengan Module http Node.js**		
Dosen Pengampu	Rizka Ardiansyah, S.Kom., M.Kom.		

A. Tujuan Praktikum (Sub-CPMK)

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

No.	Tujuan Pembelajaran Praktikum	Level
1	Menjelaskan prinsip arsitektur REST, meliputi resource, representation, stateless, dan uniform interface.	**C2**
2	Mengidentifikasi komponen utama RESTful API, yaitu endpoint, HTTP method, dan status code.	**C2**
3	Membangun HTTP server menggunakan module http bawaan Node.js tanpa framework.	**C3**
4	Mengimplementasikan endpoint CRUD dengan pemetaan HTTP method dan status code yang tepat.	**C3**
5	Menganalisis kelebihan dan keterbatasan RESTful API berdasarkan pengalaman implementasi.	**C4**

Pemetaan Capaian Pembelajaran

Kode	Rumusan Capaian Pembelajaran
CPL-02	Menerapkan (C3), menganalisis (C4), mengevaluasi (C5), dan merancang (C6) algoritma, struktur data, serta layanan API untuk menyelesaikan permasalahan komputasional dan mendukung pengembangan aplikasi pada platform khusus (IoT, blockchain, AR/VR, dan metaverse) secara efisien, serta menunjukkan kepedulian (A2) dalam pemecahan persoalan di lingkungan masyarakat.

Kode	Rumusan Capaian Pembelajaran
CPMK-1	Mahasiswa mampu menjelaskan prinsip, konsep dasar, dan peran API dalam integrasi sistem, termasuk perbedaan pendekatan RESTful dan GraphQL, serta memahami aspek keamanan dan manajemen akses API.
Sub-CPMK 2	Membedakan arsitektur RESTful dan GraphQL serta menjelaskan kelebihan dan kekurangannya.
CPMK-2	Mahasiswa mampu merancang, mengimplementasikan, mendokumentasikan, serta menguji API berbasis RESTful maupun GraphQL sesuai standar industri, dengan memperhatikan keamanan, pengelolaan versi, dan pemeliharaan.
Sub-CPMK 4 (pendukung)	Mengimplementasikan dan menguji API sederhana berbasis RESTful/GraphQL.
Posisi dalam RPS	Pertemuan ke-4, Konsep RESTful API, dengan sasaran utama Sub-CPMK 2. Implementasi pada modul ini sekaligus menjadi fondasi teknis proyek kelompok Evaluasi Sub-CPMK 4 yang didemonstrasikan pada Pertemuan ke-7. Codebase yang dibangun di sini terus dikembangkan pada praktikum-praktikum berikutnya.

Alokasi Waktu

Tahapan Kegiatan	Alokasi Waktu
Ceramah interaktif konsep REST dan studi kasus dokumentasi RESTful API	40 menit
Pre-Test	10 menit
Pelaksanaan Praktikum (C.1 – C.8)	130 menit
Post-Test dan penutup	20 menit
Total Alokasi Waktu (sesuai RPS Pertemuan ke-4)	**4 × 50 menit**

B. Landasan Teori

B.1 Berpindah Sisi: dari Consumer ke Provider

Dua praktikum sebelumnya menempatkan mahasiswa di sisi client. Pada Praktikum 1 mahasiswa mengamati percakapan HTTP melalui Postman dan curl, sedangkan pada Praktikum 2 mahasiswa memanggil API dari program Node.js. Mulai praktikum ini posisinya berbalik. Mahasiswa menjadi **API provider**, yakni pihak yang menunggu request datang lalu menyusun jawabannya. Baris status, header, dan body yang dahulu hanya dibaca kini harus dihasilkan sendiri.

Praktikum ini memakai konvensi penamaan campuran yang lazim dipakai di industri. Nama resource pada endpoint dan nama field pada JSON ditulis dalam bahasa Inggris, karena keduanya merupakan kontrak publik yang dibaca pihak lain. Nama fungsi dan variable internal ditulis dalam bahasa Indonesia agar alur logika mudah diikuti.

B.2 REST sebagai Gaya Arsitektur

REST (Representational State Transfer) diperkenalkan oleh Roy Fielding dalam disertasinya pada tahun 2000. REST bukan protokol dan bukan standar yang memiliki spesifikasi baku, melainkan **gaya arsitektur** (architectural style), yaitu sekumpulan **constraint** yang apabila dipatuhi menghasilkan sistem yang sederhana, dapat diskalakan, dan mudah dikembangkan secara terpisah. Constraint utamanya meliputi pemisahan tegas antara client dan server, komunikasi yang bersifat stateless, response yang dapat di-cache, sistem berlapis (layered system), dan **uniform interface**.

Konsep sentral REST adalah **resource**, yaitu segala sesuatu yang dapat diberi nama, misalnya sebuah destinasi wisata atau daftar seluruh destinasi. Setiap resource diidentifikasi oleh URI seperti /destinations/3. Client tidak pernah menerima resource itu sendiri, melainkan **representation** dari resource tersebut, yang pada praktikum ini berbentuk JSON. Resource yang sama dapat saja dikirim dalam representation lain, misalnya XML, tanpa mengubah identitasnya.

Constraint **uniform interface** berarti seluruh resource dimanipulasi dengan cara yang sama, yaitu melalui HTTP method yang maknanya sudah disepakati. Akibatnya, pengembang yang memahami cara memakai /destinations dapat langsung menebak cara memakai /users tanpa membaca dokumentasi secara mendalam. Karena itu penamaan endpoint mengikuti pola yang ketat: memakai kata benda jamak, bukan kata kerja. Endpoint /destinations sudah benar, sedangkan /getDestinations menyalahi pola karena tindakan sudah dinyatakan oleh method GET.

B.3 Pemetaan Operasi ke HTTP Method dan Status Code

Operasi dasar terhadap data, yang lazim disingkat CRUD (create, read, update, delete), dipetakan ke HTTP method beserta status code yang sesuai sebagaimana disajikan pada Tabel 3.1.

Operasi	Method dan Endpoint	Status Berhasil	Status Gagal
Membaca seluruh data	`GET /destinations`	200 OK	—
Membaca satu data	`GET /destinations/{id}`	200 OK	400, 404
Membuat data baru	`POST /destinations`	201 Created	400
Mengganti data	`PUT /destinations/{id}`	200 OK	400, 404
Menghapus data	`DELETE /destinations/{id}`	204 No Content	404
Method tidak didukung	`misalnya PATCH /destinations`	—	405 dengan header Allow

Tabel 3.1 Pemetaan operasi CRUD ke HTTP method dan status code

Dua status pada tabel tersebut memiliki kewajiban tambahan. Status **201 Created** sebaiknya disertai header **Location** yang menunjuk alamat resource yang baru dibuat. Status **405 Method Not Allowed** wajib disertai header **Allow** yang menyebutkan method apa saja yang didukung endpoint tersebut. Sementara itu, status 204 berarti operasi berhasil tetapi server sengaja tidak mengirim body.

Konsep lain yang perlu dipahami adalah **idempotent**. Sebuah method bersifat idempotent apabila memanggilnya berkali-kali menghasilkan keadaan akhir yang sama dengan memanggilnya sekali. GET, PUT, dan DELETE bersifat idempotent, sedangkan POST tidak, karena setiap pemanggilan POST

menciptakan resource baru. Sifat ini menentukan apakah sebuah request aman diulang secara otomatis ketika koneksi terputus di tengah jalan.

B.4 Anatomi Server dengan Module http

Module http menyediakan fungsi **createServer** yang menerima sebuah fungsi handler. Handler tersebut dipanggil satu kali untuk setiap request yang masuk dan menerima dua object. Object **req** memuat informasi request seperti method, url, dan headers, sedangkan body request tiba sebagai stream berisi chunk, persis seperti body response pada Praktikum 2, hanya kini arahnya masuk. Object **res** dipakai untuk menyusun jawaban melalui statusCode, setHeader, dan end. Pemanggilan end mengirim jawaban sekaligus menutup percakapan. Handler yang lupa memanggil end membuat client menunggu tanpa kepastian.

Module http tidak menyediakan routing. Tidak ada mekanisme otomatis yang mengarahkan GET /destinations ke satu fungsi dan DELETE /destinations/3 ke fungsi lain. **Routing** pada dasarnya hanyalah rangkaian keputusan berdasarkan method dan path, dan pada praktikum ini keputusan itu ditulis sendiri menggunakan if. Framework seperti Express pada intinya membungkus pekerjaan yang sama dengan sintaks yang lebih ringkas.

B.5 Kelebihan dan Keterbatasan REST

REST menjadi gaya arsitektur API yang paling banyak dipakai karena beberapa alasan. Interface-nya seragam sehingga mudah dipelajari. REST memanfaatkan HTTP secara penuh, termasuk status code dan mekanisme cache pada method GET. Sifat stateless memudahkan server diperbanyak untuk menampung beban tinggi. Dukungan tools dan library untuk REST juga tersedia di hampir semua bahasa pemrograman.

Keterbatasannya muncul ketika kebutuhan client beragam. Endpoint REST mengembalikan bentuk data yang ditentukan server, sehingga client yang hanya membutuhkan nama destinasi tetap menerima seluruh field. Gejala ini disebut **over-fetching**. Sebaliknya, client yang membutuhkan destinasi beserta ulasan dan informasi daerahnya terpaksa memanggil beberapa endpoint secara berurutan, yang disebut **under-fetching**. Upaya menambal kedua masalah tersebut dengan membuat endpoint khusus untuk setiap kebutuhan akan membuat jumlah endpoint terus bertambah. Keterbatasan ini bukan cacat, melainkan konsekuensi dari pilihan desain, dan justru menjadi alasan lahirnya GraphQL yang dibahas pada praktikum berikutnya.

C. Implementasi Praktikum

Resource yang dibangun pada praktikum ini adalah data destinasi wisata di Sulawesi Tengah. Data disimpan dalam file JSON menggunakan module fs, sehingga tetap tersedia meskipun server dimatikan dan dinyalakan kembali. Seluruh kode hanya memakai core module Node.js.

C.1 Server Pertama

Buat folder kerja baru dan inisialisasi project seperti pada Praktikum 2.

```
mkdir praktikum-3
cd praktikum-3
npm init -y
```

Kode 3.1 Inisialisasi project praktikum

Buka folder praktikum-3 menggunakan text editor, lalu buat file server-awal.js dengan isi berikut.

```
const http = require('http');
```

```
const server = http.createServer((req, res) => {
  console.log(req.method, req.url);
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain; charset=utf-8');
  res.end('Halo dari server Node.js pertama saya');
});

server.listen(3000, () => {
  console.log('Server berjalan di http://localhost:3000');
});
```

Kode 3.2 Server HTTP paling sederhana

Jalankan dengan perintah node server-awal.js. Terminal tidak kembali ke prompt karena server kini menunggu request. Buka http://localhost:3000 pada browser, lalu buka terminal kedua dan jalankan perintah curl -v http://localhost:3000. Pengguna Windows PowerShell mengetik curl.exe sebagai pengganti curl.

```
> GET / HTTP/1.1
> Host: localhost:3000
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/1.1 200 OK
< Content-Type: text/plain; charset=utf-8
< Date: Mon, 21 Sep 2026 12:20:32 GMT
< Connection: keep-alive
< Content-Length: 37
<
Halo dari server Node.js pertama saya
```

Kode 3.3 Percakapan HTTP yang kini dihasilkan server buatan sendiri

Bandingkan dengan Kode 1.11 pada Praktikum 1. Susunannya identik, tetapi kali ini baris yang diawali tanda < dihasilkan oleh kode Anda sendiri. Perhatikan pula terminal pertama yang mencatat setiap request masuk. Selain GET /, Anda kemungkinan melihat GET /favicon.ico. Request tersebut dikirim otomatis oleh browser untuk mencari ikon halaman, bukan kesalahan kode Anda. Hentikan server dengan menekan Ctrl+C. Server perlu dijalankan ulang setiap kali kode diubah; untuk kenyamanan, gunakan perintah node --watch server.js yang merupakan fitur bawaan Node.js dan akan menjalankan ulang server secara otomatis.

Apabila muncul error EADDRINUSE. Pesan ini berarti port 3000 masih dipakai oleh server lain, biasanya server-awal.js yang belum dihentikan di terminal lain. Hentikan server tersebut dengan Ctrl+C, lalu jalankan ulang.

C.2 Mengirim Response JSON

Buat file server.js. File ini akan dibangun bertahap sepanjang praktikum. Awali dengan pemanggilan module, konstanta, serta dua helper function untuk mengirim response JSON.

```
const http = require('http');
const fs = require('fs/promises');
const path = require('path');

const PORT = 3000;
const DATA_FILE = path.join(__dirname, 'data', 'destinations.json');

// ---------- helper respons ----------
function kirimJSON(res, statusCode, data) {
  const body = JSON.stringify(data);
  res.writeHead(statusCode, {
    'Content-Type': 'application/json; charset=utf-8',
```

```
    'Content-Length': Buffer.byteLength(body)
  });
  res.end(body);
}

function kirimError(res, statusCode, pesan) {
  kirimJSON(res, statusCode, { error: pesan });
}
```

Kode 3.4 Pemanggilan module, konstanta, dan helper function untuk response

Fungsi kirimJSON memusatkan tiga pekerjaan yang harus dilakukan setiap kali mengirim JSON: mengubah object menjadi teks, menyetel header, lalu mengakhiri response. Header Content-Type memberi tahu client bahwa body berformat JSON. Header Content-Length dihitung dengan Buffer.byteLength, bukan panjang string, karena karakter seperti huruf beraksen atau simbol derajat membutuhkan lebih dari satu byte. Fungsi kirimError memastikan seluruh error message memiliki bentuk yang seragam, yaitu object dengan satu field error.

C.3 Menyiapkan Data dan File-based Persistence

Buat folder data di dalam folder project, lalu buat file data/destinations.json berisi data awal berikut.

```
[
  { "id": 1, "name": "Danau Poso", "region": "Poso", "category": "danau" },
  { "id": 2, "name": "Pantai Tanjung Karang", "region": "Donggala", "category": "pantai"
},
  { "id": 3, "name": "Kepulauan Togean", "region": "Tojo Una-Una", "category": "bahari" },
  { "id": 4, "name": "Taman Nasional Lore Lindu", "region": "Sigi", "category": "alam" },
  { "id": 5, "name": "Pantai Talise", "region": "Palu", "category": "pantai" }
]
```

Kode 3.5 Data awal destinasi wisata

Tambahkan dua fungsi berikut pada server.js, tepat di bawah Kode 3.4.

```
// ---------- helper data ----------
async function bacaData() {
  const isi = await fs.readFile(DATA_FILE, 'utf8');
  return JSON.parse(isi);
}

async function simpanData(data) {
  await fs.writeFile(DATA_FILE, JSON.stringify(data, null, 2));
}
```

Kode 3.6 Fungsi membaca dan menyimpan data ke file JSON

Module fs/promises menyediakan versi fungsi pengolah file yang dapat dipakai bersama await. Argumen null dan 2 pada JSON.stringify membuat isi file tersusun rapi dengan indentasi dua spasi, sehingga perubahan data mudah diamati. Pendekatan ini memadai untuk pembelajaran, tetapi memiliki keterbatasan nyata: apabila dua request menulis data pada saat yang hampir bersamaan, salah satu perubahan dapat tertimpa. Sistem produksi menyerahkan persoalan ini kepada database.

C.4 Routing Manual

Sebelum menulis fungsi untuk setiap endpoint, susun lebih dahulu kerangka routing beserta server. Letakkan kode berikut di **bagian paling bawah** server.js. Kode pada bagian C.5 hingga C.7 nantinya disisipkan di atas kerangka ini.

```
// ---------- router ----------
async function router(req, res) {
  const url = new URL(req.url, 'http://localhost');
```

```
  const segmen = url.pathname.split('/').filter(Boolean);

  if (segmen[0] !== 'destinations' || segmen.length > 2) {
    return kirimError(res, 404, 'Endpoint tidak ditemukan');
  }

  // Endpoint koleksi: /destinations
  if (segmen.length === 1) {
    if (req.method === 'GET') return daftarDestinasi(req, res, url);
    if (req.method === 'POST') return tambahDestinasi(req, res);
    res.setHeader('Allow', 'GET, POST');
    return kirimError(res, 405, `Method ${req.method} tidak didukung pada endpoint ini`);
  }

  // Endpoint item: /destinations/:id
  const id = Number(segmen[1]);
  if (!Number.isInteger(id) || id < 1) {
    return kirimError(res, 400, 'id harus berupa bilangan bulat positif');
  }

  if (req.method === 'GET') return detailDestinasi(req, res, id);
  if (req.method === 'PUT') return ubahDestinasi(req, res, id);
  if (req.method === 'DELETE') return hapusDestinasi(req, res, id);
  res.setHeader('Allow', 'GET, PUT, DELETE');
  return kirimError(res, 405, `Method ${req.method} tidak didukung pada endpoint ini`);
}

// ---------- server ----------
const server = http.createServer(async (req, res) => {
  console.log(`${new Date().toISOString()}  ${req.method} ${req.url}`);
  try {
    await router(req, res);
  } catch (err) {
    if (err.statusCode) {
      return kirimError(res, err.statusCode, err.message);
    }
    console.error(err);
    kirimError(res, 500, 'Terjadi kesalahan pada server');
  }
});

server.listen(PORT, () => {
  console.log(`Server berjalan di http://localhost:${PORT}`);
});
```

Kode 3.7 Kerangka routing dan server

Object URL memecah alamat request menjadi path dan query parameter. Path kemudian dipecah menjadi segmen, sehingga /destinations/3 menjadi array berisi destinations dan 3. Jumlah segmen menentukan apakah request ditujukan ke endpoint collection atau endpoint item. Nilai id diubah menjadi angka dan divalidasi lebih dahulu, sehingga fungsi handler tidak perlu mengkhawatirkan id yang tidak sah. Setiap method yang tidak didukung dijawab 405 beserta header Allow.

Blok try/catch pada server membedakan dua jenis error. Error yang disengaja, yaitu yang membawa properti statusCode, diteruskan kepada client apa adanya. Error tak terduga dicatat lengkap di terminal server, tetapi client hanya menerima pesan umum dengan status 500. Detail internal seperti lokasi file dan baris kode tidak boleh bocor kepada client, sebuah prinsip yang akan diperdalam pada praktikum keamanan API.

Kerangka ini sudah dapat diuji meskipun fungsi handler belum ditulis. Jalankan server dengan perintah node --watch server.js, lalu kirim tiga request berikut melalui Postman: GET http://localhost:3000/wisata, PATCH http://localhost:3000/destinations, dan GET http://localhost:3000/destinations/abc. Ketiganya

harus menghasilkan status 404, 405, dan 400. Endpoint lain belum dapat diuji sebelum bagian berikutnya diselesaikan.

C.5 Endpoint GET

Sisipkan dua fungsi handler berikut di atas kerangka routing.

```
// ---------- handler ----------
async function daftarDestinasi(req, res, url) {
  let data = await bacaData();
  const kategori = url.searchParams.get('category');
  if (kategori) {
    data = data.filter((item) => item.category === kategori);
  }
  kirimJSON(res, 200, data);
}

async function detailDestinasi(req, res, id) {
  const data = await bacaData();
  const item = data.find((d) => d.id === id);
  if (!item) {
    return kirimError(res, 404, `Destinasi dengan id ${id} tidak ditemukan`);
  }
  kirimJSON(res, 200, item);
}
```

Kode 3.8 Handler untuk membaca seluruh data dan satu data

Fungsi daftarDestinasi membaca query parameter category melalui url.searchParams. Query parameter yang pada Praktikum 1 Anda kirim ke server orang lain kini Anda terima dan tafsirkan sendiri. Server yang dijalankan dengan node --watch otomatis memuat ulang kode baru; apabila Anda memakai node biasa, hentikan lalu jalankan ulang server. Setelah itu buka alamat http://localhost:3000/destinations, http://localhost:3000/destinations?category=pantai, http://localhost:3000/destinations/2, dan http://localhost:3000/destinations/99 melalui browser maupun Postman. Tiga alamat pertama menghasilkan status 200 dan alamat terakhir menghasilkan 404.

C.6 Membaca Body Request dan Endpoint POST

Request POST membawa data di dalam body. Tambahkan helper function berikut tepat di bawah Kode 3.6.

```
// ---------- helper request ----------
const FIELD_WAJIB = ['name', 'region', 'category'];

function buatError(statusCode, pesan) {
  const err = new Error(pesan);
  err.statusCode = statusCode;
  return err;
}

function bacaBody(req) {
  return new Promise((resolve, reject) => {
    let body = '';
    req.on('data', (chunk) => { body += chunk; });
    req.on('end', () => {
      if (body === '') return resolve(null);
      try {
        resolve(JSON.parse(body));
      } catch (err) {
        reject(buatError(400, 'Body request bukan JSON yang valid'));
      }
    });
```

```
    req.on('error', reject);
  });
}

function cekFieldWajib(input) {
  if (input === null || typeof input !== 'object' || Array.isArray(input)) {
    return FIELD_WAJIB;
  }
  return FIELD_WAJIB.filter(
    (field) => typeof input[field] !== 'string' || input[field].trim() === ''
  );
}
```

Kode 3.9 Helper function untuk membaca dan memvalidasi body request

Fungsi bacaBody adalah cerminan dari Kode 2.2 pada Praktikum 2. Dahulu Anda merakit chunk body response yang datang dari server lain, kini Anda merakit chunk body request yang datang dari client. Mekanismenya persis sama. Body yang bukan JSON sah ditolak dengan buatError berstatus 400. Fungsi cekFieldWajib mengembalikan daftar field yang belum terisi dengan benar, sehingga error message dapat menyebutkan secara tepat apa yang kurang.

Selanjutnya sisipkan handler POST di bawah Kode 3.8.

```
async function tambahDestinasi(req, res) {
  const input = await bacaBody(req);
  const kurang = cekFieldWajib(input);
  if (kurang.length > 0) {
    return kirimError(res, 400, `Field wajib belum diisi: ${kurang.join(', ')}`);
  }

  const data = await bacaData();
  const idBaru = data.length === 0 ? 1 : Math.max(...data.map((d) => d.id)) + 1;
  const item = {
    id: idBaru,
    name: input.name.trim(),
    region: input.region.trim(),
    category: input.category.trim()
  };

  data.push(item);
  await simpanData(data);

  res.setHeader('Location', `/destinations/${idBaru}`);
  kirimJSON(res, 201, item);
}
```

Kode 3.10 Handler untuk membuat data baru

Nilai id baru ditentukan server, bukan client, dengan mengambil id terbesar ditambah satu. Setiap nilai teks dirapikan menggunakan trim sebelum disimpan. Response dikirim dengan status 201 beserta header Location yang menunjuk alamat resource baru.

C.7 Endpoint PUT dan DELETE

Sisipkan dua handler terakhir berikut di bawah Kode 3.10.

```
async function ubahDestinasi(req, res, id) {
  const input = await bacaBody(req);
  const data = await bacaData();
  const index = data.findIndex((d) => d.id === id);
  if (index === -1) {
    return kirimError(res, 404, `Destinasi dengan id ${id} tidak ditemukan`);
  }
```

```
  const kurang = cekFieldWajib(input);
  if (kurang.length > 0) {
    return kirimError(res, 400, `Field wajib belum diisi: ${kurang.join(', ')}`);
  }

  data[index] = {
    id,
    name: input.name.trim(),
    region: input.region.trim(),
    category: input.category.trim()
  };
  await simpanData(data);
  kirimJSON(res, 200, data[index]);
}

async function hapusDestinasi(req, res, id) {
  const data = await bacaData();
  const index = data.findIndex((d) => d.id === id);
  if (index === -1) {
    return kirimError(res, 404, `Destinasi dengan id ${id} tidak ditemukan`);
  }

  data.splice(index, 1);
  await simpanData(data);
  res.writeHead(204);
  res.end();
}
```

Kode 3.11 Handler untuk mengganti dan menghapus data

Method PUT mengganti seluruh isi resource, sehingga seluruh field wajib tetap harus dikirim meskipun hanya satu yang berubah. Pemeriksaan keberadaan data dilakukan sebelum validasi, karena resource yang tidak ada lebih tepat dijawab 404 daripada 400. Method DELETE menjawab dengan status 204 tanpa body. Pada titik ini file server.js telah lengkap.

C.8 Pengujian dengan Postman

Pada collection Praktikum Rekayasa API, buat folder baru bernama Praktikum 3. Buka tab Variables pada collection, lalu tambahkan variable baru bernama local_url dengan nilai http://localhost:3000. Variable base_url dari Praktikum 1 tetap dibiarkan karena masih menunjuk GitHub API. Sebelum mulai, kembalikan isi file data/destinations.json persis seperti Kode 3.5 agar hasil pengujian sesuai tabel. Jalankan ketiga belas request pada Tabel 3.2 secara berurutan, karena beberapa request bergantung pada hasil request sebelumnya. Untuk request yang membawa body, buka tab Body, pilih opsi raw, lalu pilih format JSON, kemudian salin isi body yang sesuai dari Kode 3.12.

No	Request	Body	Status yang Diharapkan
1	`GET {{local_url}}/destinations`	—	200
2	`GET {{local_url}}/destinations?category=pantai`	—	200, dua data
3	`GET {{local_url}}/destinations/2`	—	200
4	`GET {{local_url}}/destinations/99`	—	404
5	`GET {{local_url}}/destinations/abc`	—	400

No	Request	Body	Status yang Diharapkan
6	`POST {{local_url}}/destinations`	Body A	201, header Location
7	`POST {{local_url}}/destinations`	Body B	400
8	`POST {{local_url}}/destinations`	Body C	400
9	`PUT {{local_url}}/destinations/6`	Body D	200
10	`DELETE {{local_url}}/destinations/6`	—	204
11	`DELETE {{local_url}}/destinations/6`	—	404
12	`PATCH {{local_url}}/destinations`	—	405, header Allow
13	`GET {{local_url}}/wisata`	—	404

Tabel 3.2 Skenario pengujian endpoint destinasi

```
Body A  (request nomor 6)
{ "name": "Air Terjun Wera", "region": "Sigi", "category": "alam" }

Body B  (request nomor 7)
{ "name": "Air Terjun Wera" }

Body C  (request nomor 8, sengaja tidak valid)
{ 'name': 'Air Terjun Wera' }

Body D  (request nomor 9)
{ "name": "Air Terjun Wera", "region": "Sigi", "category": "air terjun" }
```

Kode 3.12 Isi body untuk request nomor 6 sampai 9

Postman akan memberi garis merah pada Body C karena isinya bukan JSON yang valid. Tetap kirim request tersebut, karena justru itulah yang sedang diuji. Request nomor 9 sampai 11 memakai id 6, yaitu id yang dibuat oleh request nomor 6 pada data awal. Periksa header Location pada response request nomor 6; apabila nilainya berbeda, gunakan id tersebut pada request nomor 9 sampai 11.

Setelah pengujian selesai, buka file data/destinations.json dan amati perubahannya. Susunan teksnya kini berbeda dari Kode 3.5 karena file ditulis ulang oleh fungsi simpanData dengan indentasi dua spasi, tetapi isi datanya kembali sama dengan data awal. Request nomor 11 menghasilkan 404 karena resource telah dihapus oleh request nomor 10, yang sekaligus membuktikan sifat idempotent DELETE: keadaan akhirnya tetap sama walaupun status code-nya berbeda. Simpan seluruh request tersebut dalam collection, karena akan dipakai kembali pada praktikum pengujian API.

D. Tugas Mahasiswa

1. Kerjakan seluruh langkah C.1 sampai C.8 dan pastikan ketiga belas skenario pada Tabel 3.2 menghasilkan status code sesuai harapan.
2. Bangun RESTful API from scratch untuk satu resource pilihan Anda sendiri yang berbeda dari destinasi wisata, sebaiknya resource yang direncanakan menjadi bagian proyek kelompok Evaluasi Sub-CPMK 4 (Tugas Besar). Gunakan server.js dari praktikum ini sebagai titik awal.

3. Resource tersebut memiliki sekurang-kurangnya empat field, termasuk sekurang-kurangnya satu field bertipe number yang divalidasi, misalnya tidak boleh bernilai negatif. Validasi tersebut dapat ditulis dengan memeriksa Number.isFinite(nilai) dan nilai >= 0.
4. Sediakan lima endpoint, yaitu GET collection, GET item, POST, PUT, dan DELETE, ditambah sekurang-kurangnya satu query parameter filter pada GET collection. Status code 200, 201, 204, 400, 404, dan 405 harus dapat dipicu.
5. Program hanya boleh memakai core module Node.js. File package.json tidak boleh memuat bagian dependencies.
6. Tambahkan satu baris console.log berisi nama lengkap dan stambuk Anda pada baris paling atas server.js, sehingga identitas tersebut tercetak setiap kali server dijalankan.
7. Susun laporan praktikum dengan sistematika berikut.
    a. Halaman sampul yang memuat judul praktikum, nama, dan stambuk.
    b. Hasil praktikum destinasi: tabel status code yang Anda peroleh untuk ketiga belas skenario pada Tabel 3.2, disertai screenshot terminal server yang memperlihatkan baris identitas dan log seluruh request.
    c. Rancangan API buatan sendiri: nama resource beserta alasan pemilihannya, daftar field lengkap dengan tipe data dan aturan validasinya, serta tabel endpoint yang memuat method, path, status berhasil, dan status gagal seperti Tabel 3.1.
    d. Implementasi: isi server.js lengkap yang disalin sebagai teks, bukan screenshot, disertai penjelasan bagian yang Anda ubah atau tambahkan dari kode praktikum.
    e. Pengujian: sekurang-kurangnya dua test case untuk setiap endpoint, yakni satu skenario berhasil dan satu skenario gagal sesuai indikator Sub-CPMK 4, disajikan dalam tabel berisi request, body, status yang diharapkan, dan status yang diperoleh, disertai screenshot Postman.
    f. Analisis maksimal satu halaman: tunjukkan satu kebutuhan client terhadap API Anda yang memaksa terjadinya over-fetching atau under-fetching, jelaskan bagaimana REST dapat mengatasinya, lalu uraikan konsekuensi solusi tersebut terhadap jumlah endpoint dan pemeliharaan API.
    g. Kesimpulan.
8. Kumpulkan laporan praktikum kepada asisten praktikum sesuai format dan batas waktu yang ditetapkan asisten.
9. Analisis pada butir 7f menjadi bahan pembanding untuk Evaluasi Sub-CPMK 2 berupa analisis perbandingan RESTful API dan GraphQL, sedangkan API yang Anda bangun menjadi fondasi proyek kelompok Evaluasi Sub-CPMK 4.

.